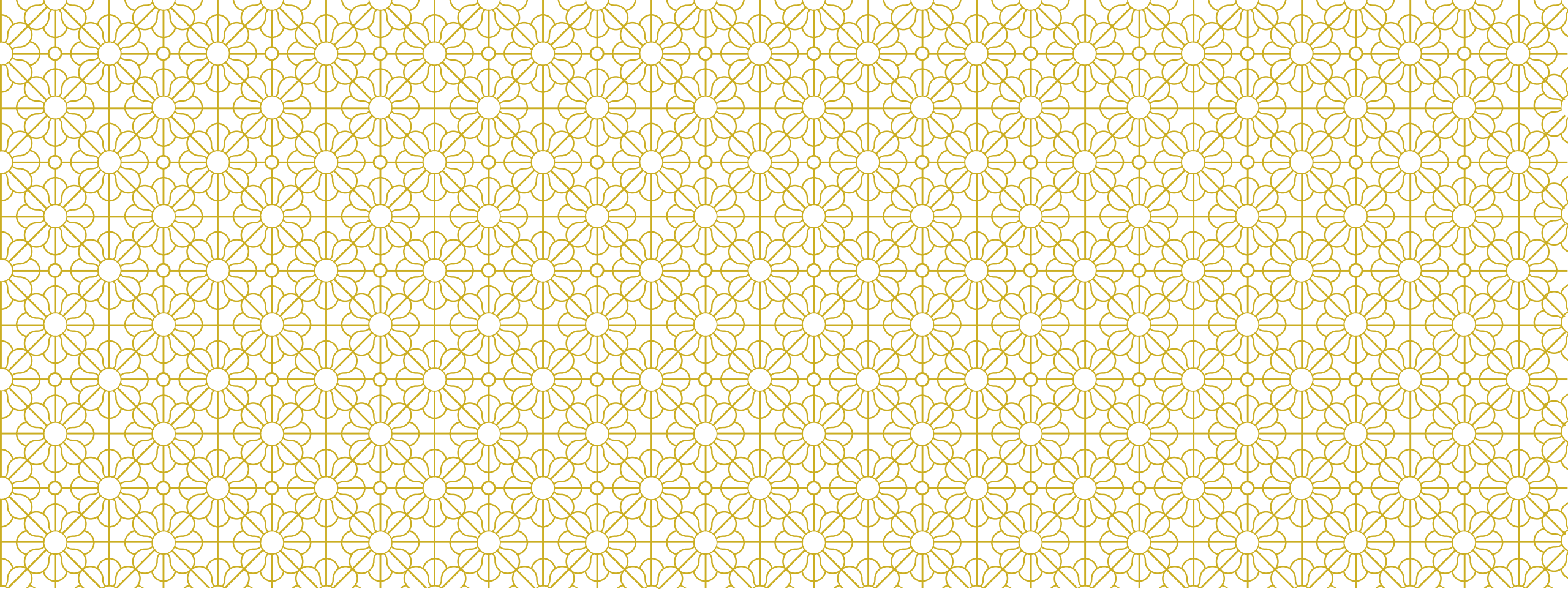




CSE 420 LECTURE 19

Runtime Engine: Stack

Background Review

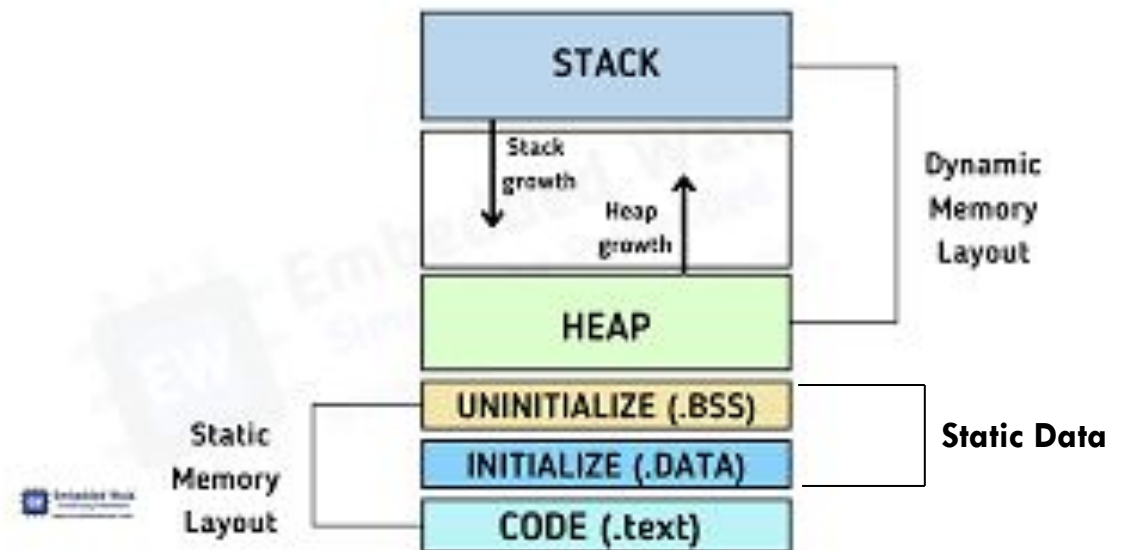
MEMORY LAYOUT OF A PROCESS

You learned about the memory layout of a process in the operating system course.

You also learned then that this layout is the layout of the logical address space of the process.

Actually, it is the compiler that is responsible for generating this layout.

By this, we understand that the compiler generates the executable machine code in a manner so that our known address space is maintained.



CONTENT OF AN EXECUTABLE PROGRAM

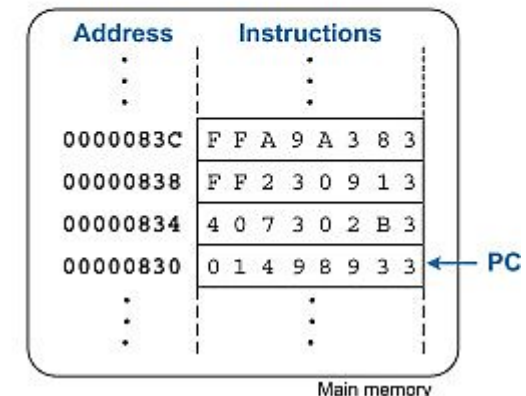
When your executable program is resting on the hard disk, it only contains the architecture-specific machine code corresponding to three-address code that we learned to generate.

Apart from that some meta-data information retains information about static data sections and some other information.

When a process is created by the operating system for your program and the process is allowed to execute in the CPU/CPU-cores then the variables of your program gets their values and their space in the logical address space.

Hence it is the compiler that decides where what variable should be allocated and how and generates the machine instructions accordingly so that values of variables can be read/write.

Assembly code	Machine code
add s2, s3, s4	0x01498933
sub t0, t1, t2	0x407302B3
addi s2, t1, -14	0xFF230913
lw t2, -6(s3)	0xFFA9A383

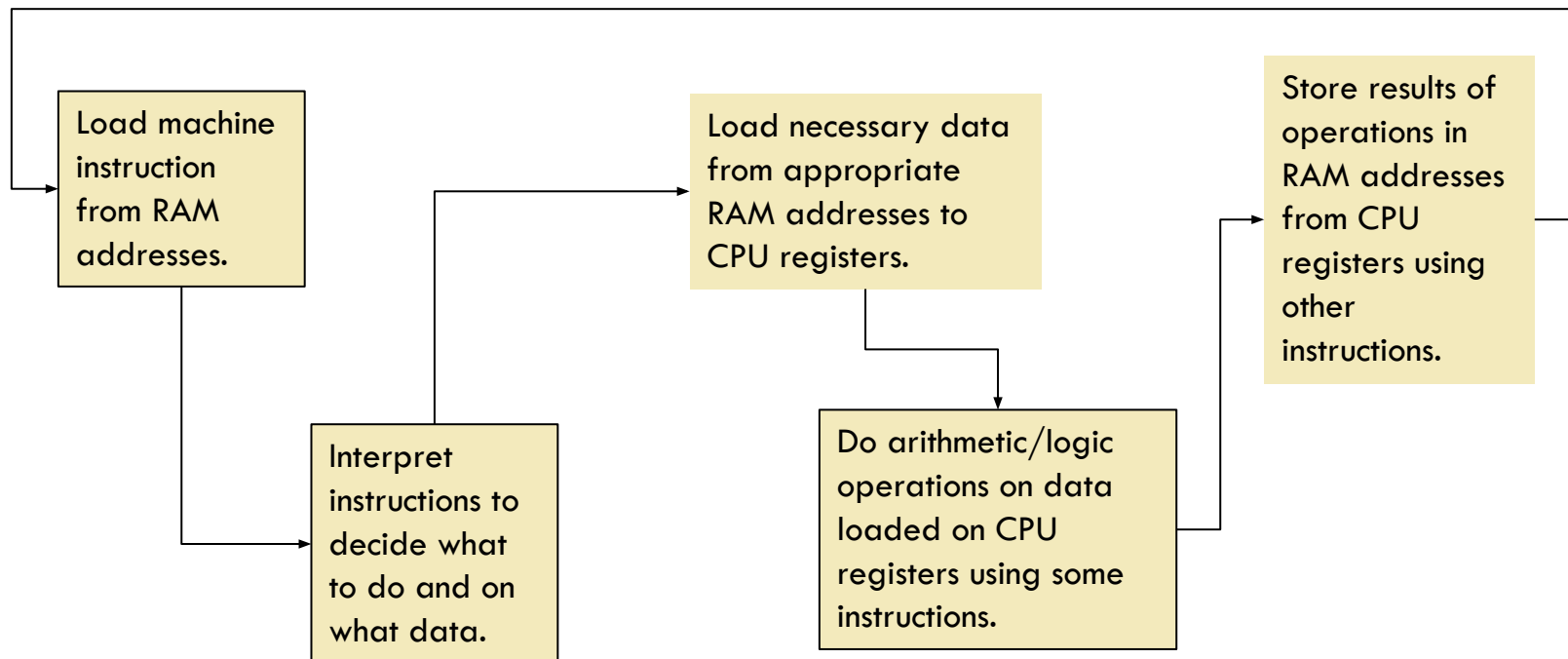


ADDRESSES NOT VARIABLES

Remember that there is no concepts of variables in machine instructions in any ISA architecture.

Furthermore, direct arithmetic/logic operations on values stored in RAM addresses are also impossible.

Rather the way a process makes progress can be described as the following:



Memory Access: Load				
Op	Rs1	ws	Offset	
4	3	3	6	
Memory Access: Store				
Op	Rs1	Rs2	Offset	
4	3	3	6	
Data Processing:				
Op	Rs1	Rs2	ws	Offset
4	3	3	3	3
Branch:				
Op	Rs1	Rs2	Offset	
4	3	3	6	
Jump:				
Op		Offset		
4		12		

OP	
0000	Load Word
0001	Store Word
0002	Add
0003	Subtract
0004	Invert
0005	Logical Shift Left
0006	Logical Shift Right
0007	Bitwise AND
0008	Bitwise OR
0009	Set on less than
0010	Hamming Distance
0011	Branch on Equal
0012	Branch on NOT Equal
0013	Jump

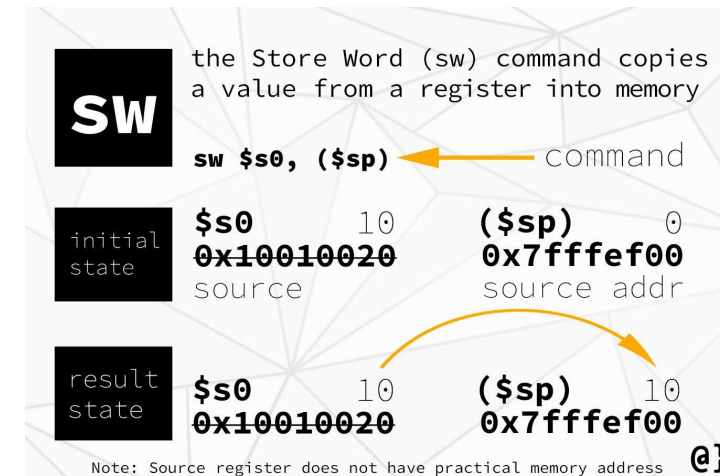
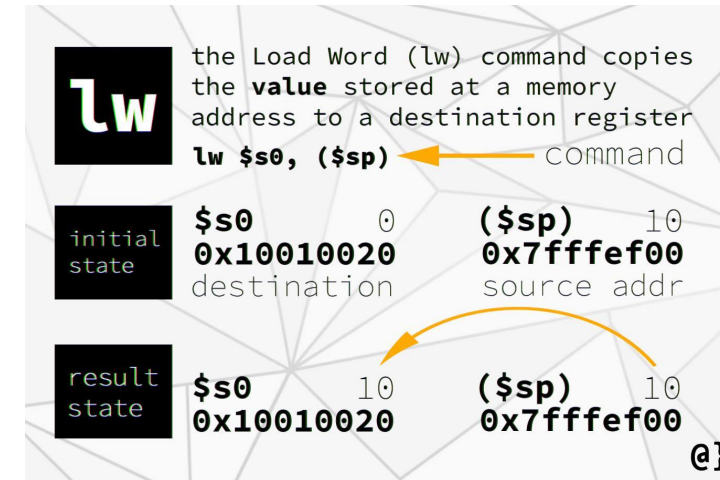
LOADING STORING VARIABLES TO/FROM CPU

During the code generation phase, compiler translates the variables and temporaries of the three-address code into logical addresses in the RAM.

All access to the variables and temporaries are then translated into accessing specific RAM addresses.

The compiler, uses the symbol tables of underlying scope to decide which address should be used for what variable.

Then some variant of load and store instructions are used to move data to/from RAM and CPU for arithmetic and logic operations.

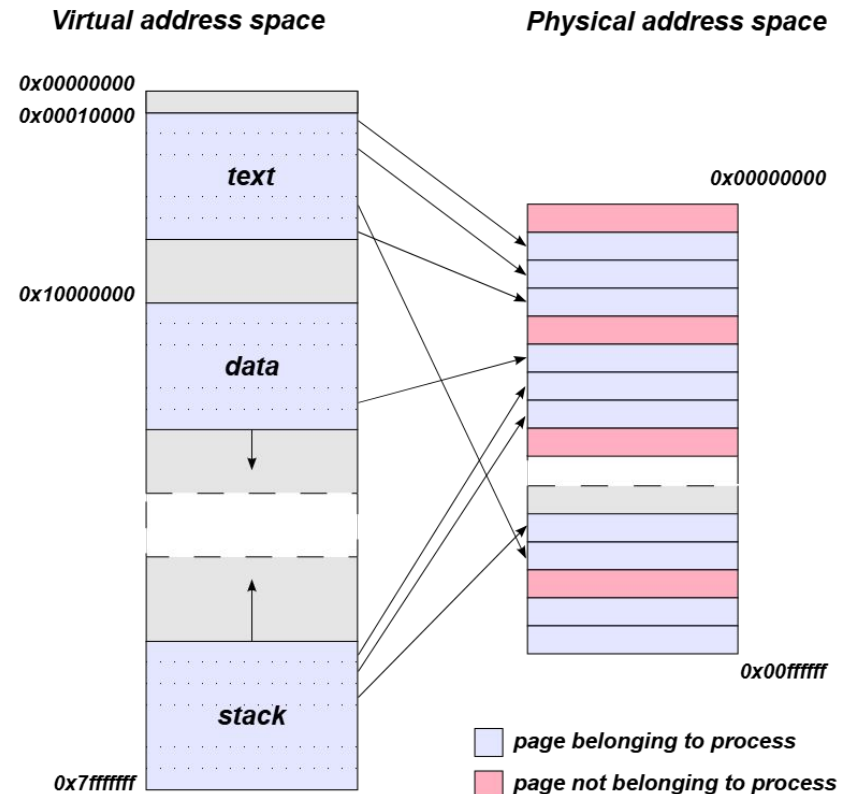


LOGICAL AND PHYSICAL ADDRESS SPACES

Remember that the addresses of instructions and data used in the machine code are logical addresses.

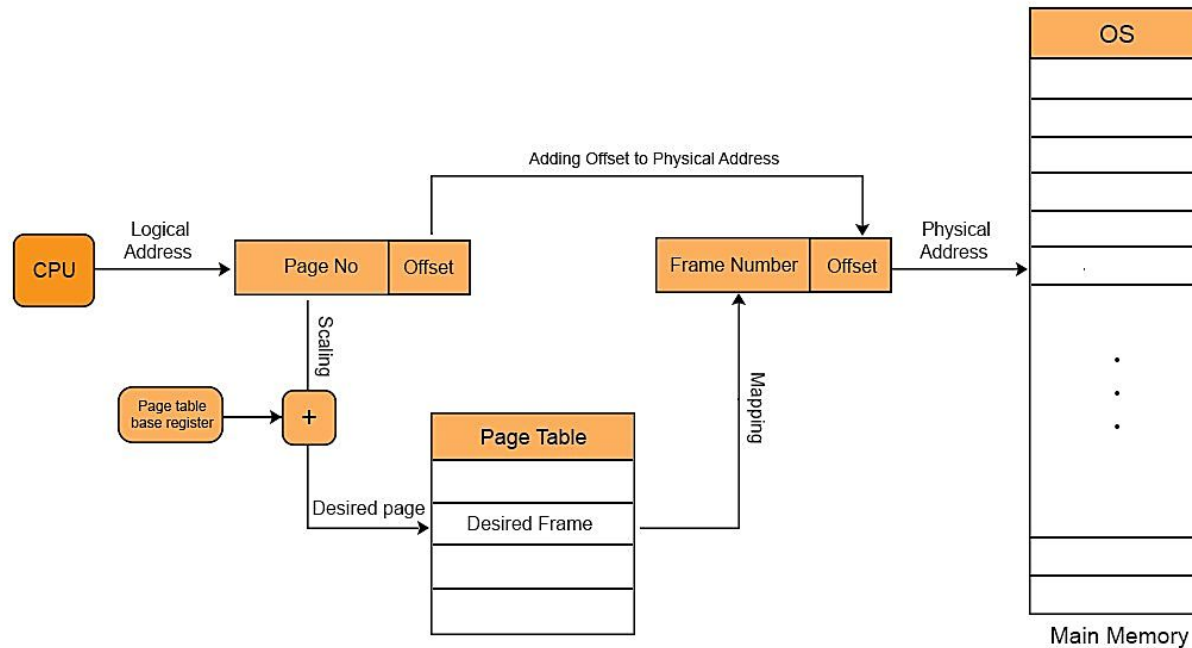
In reality, operating system divides the logical address space into equal size chunks, called pages, and give different physical chunks, called frames, in the RAM to this different chunks.

This process is called logical-to-physical address mapping and the process in execution knows nothing about this difference.



COMPILER AND OPERATING SYSTEM ROLE BREAKDOWN

During the process execution, CPU hardware automatically translate logical addresses of instructions and data to proper physical addresses.



The operating system is only responsible for allocating frames for pages and loading some metadata in specific CPU registers so that the translation can be done by the CPU.

So as long as the compiler generates instructions that refer to proper logical addresses, everything should be alright.

Memory Layout and Machine Code Interplay

ASSIGNING ADDRESSES TO VARIABLES

For global and fixed length variables, address deciding is easy.

The compiler fixes specific locations in the static data section for variables.

Symbol table information is used to determine how much space each variable is needed and in what order to assign them address to avoid space overlapping.

Subsequently, three address instructions referring to the variables are replaced by access of those addresses.

Deciding the addresses of function parameters and local variables is much more complicated, for which we need the **stack**.

EXAMPLE OF GLOBAL VARIABLE ADDRESS ASSIGNMENT

Source code:

```
int x = 10;  
float y;  
  
void main() {  
    y = x * 2.5 + 1.6;  
}
```

Three Address code:

```
t0 = x * 2.5  
t1 = t0 + 1.6  
y = t1
```

Assembly Code:

```
lw $s1, $s0[0]  
multi $t0, $s1, 2.5  
Addi $t1, $t0, 1.6  
sdw $t1, $s0[4]
```

Here \$.. Means some CPU Register.

Static Data

Text

Address for y

8 bytes

Address for x

4 bytes

Machine Instructions

Layout of the logical Address Space

Assume the address where text section end is stored in register \$s0

CHALLENGES WITH FUNCTION LOCAL VARIABLES

Static address allocation of variables does not work for variables that are local to functions.

This fact can be understood from nested function calls.

However, recursive functions illustrates this concepts very well.

```
void main() {  
    int x = 5;  
    int y = foo(x);  
    y = y / x;  
}  
  
int foo(int a) {  
    int b = a * a;  
    int c = a + a;  
    if (c < b / 2) {  
        return b + c + foo(a - 1);  
    } else {  
        return b - c;  
    }  
}
```

Consider this sample recursive code

Assumed
static
addresses
for local
variable b
and c of
function foo

b = 25 then 16
c = 10 then 8

Main
↓
x = 5
↓
foo(4)
↓
b = 25
c = 10
↓
foo(9)
↓
b = 16
c = 8

The problem if the call from foo(5) to foo(4) will override b and c from 25 and 10 to 16 and 8 if the address is static. So we cannot go back to foo(5) and return 25 + 10 + 8 to main.

THE IDEA OF FUNCTION CALL STACK

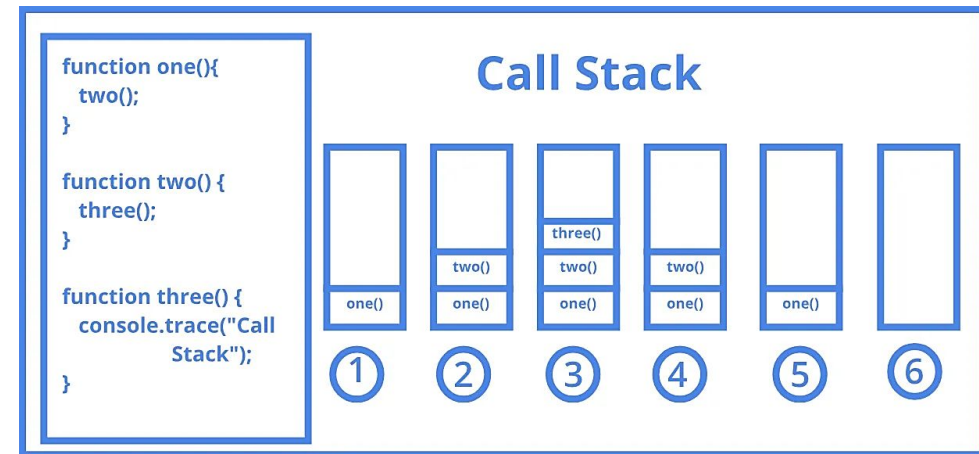
The lesson we learn from the previous example is that each time a function call happens, we must find new space in the logical address space of the process to allocate its local variables.

This gives rise to the concepts of stacks.

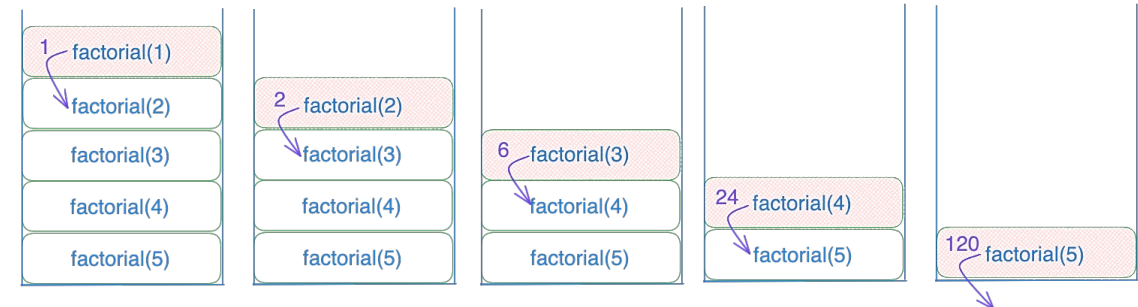
All necessary information related to function calls: including function parameters, location for return values, function's local variables, and instruction of the caller to return to when a function completes executions is stored in a stack. This information is called **function activation record**.

As function calls other functions, stack grows.

As the last called function returns, its space in the stack is freed and the code returns to the previous function using information stored in the stack.



Function activation records for nested function calls.

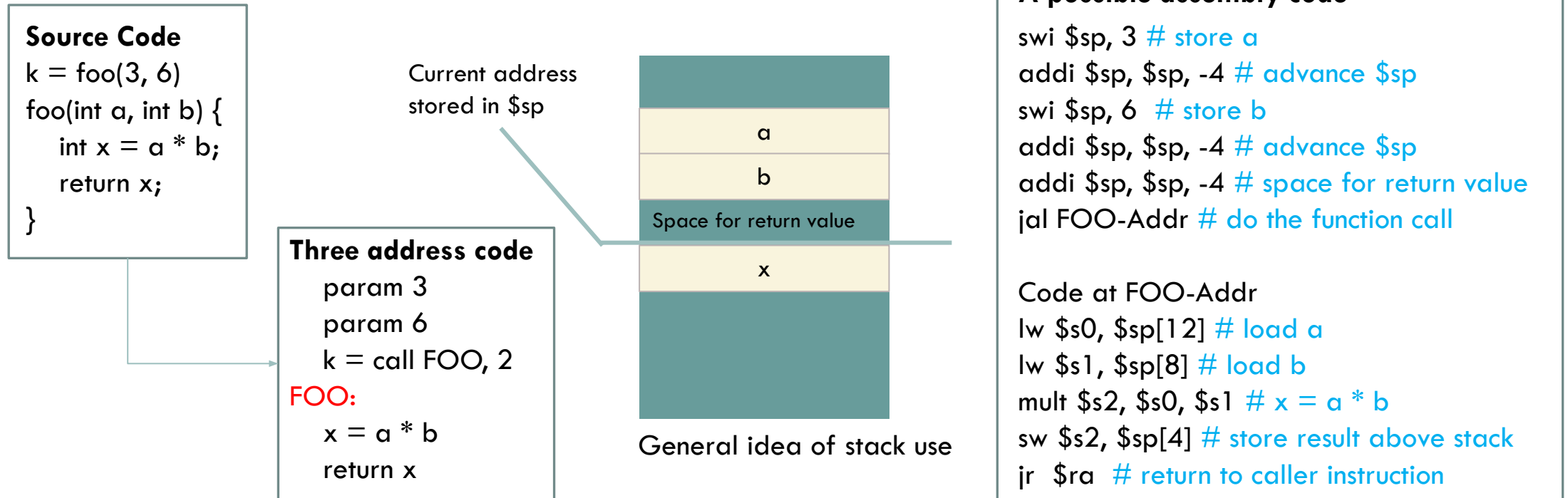


Function activation records for recursive function calls.

STACK RELATIVE VARIABLE ADDRESSING

A specific register in the CPU, called the stack pointer (denoted by \$sp) refers to the beginning of a function's local address space (where it stores local variables and temporaries). The local variables of the functions are accessed relative to that stack pointer.

This allows difference instances of the same function to use different addresses for the same local variables.



STACK SPACE ALLOCATION DESPITE HAVING PARAMETER AND RETURN VALUE REGISTERS

Most instruction set architectures have a few parameter and return value registers to pass arguments and return values between function calls.

However, the common practice is still storing them in stack because these registers only allow limited information passing.

A user defined function may have many parameters of different types that may not fit in just one or two registers.

Similarly, the return value may be so large that it may not fit in a register.

Consequently, reserving stack space is the only general options.

Compilers use these registers when they can be used for specific functions as an optimization of the general function call handling convention.

Register name	Symbolic name	Description	Saved by
32 integer registers			
x0	Zero	Always zero	
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	
x4	tp	Thread pointer	
x5	t0	Temporary / alternate return address	Caller
x6–7	t1–2	Temporary	Caller
x8	s0/fp	Saved register / frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function argument / return value	Caller
x12–17	a2–7	Function argument	Caller
x18–27	s2–11	Saved register	Callee
x28–31	t3–6	Temporary	Caller

ADVANTAGE OF STACK IN PARTICULAR

Note that for function call management, any strategy that allocates separate non-overlapping regions from the logical address space for each function should work.

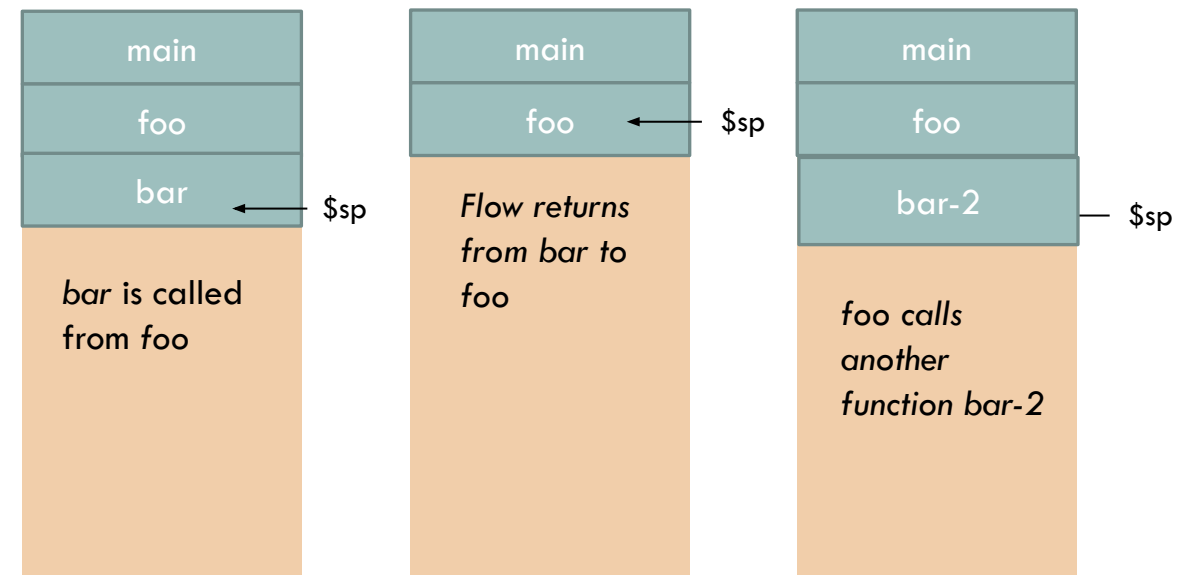
However, there is a great advantage in stack and stack relative addressing in particular.

Function call sequence has the nature of last-in-first out progression. Where, the last function being called is the one whose memory should be reclaimed first.

Afterwards, the immediately previous function that called it become active and candidate for the next cleanup.

Hence, the behavior of function call chain is just like the behavior of a stack.

The memory used by a function that flow of control returns from is reclaimed by simply decreasing the stack pointer register to proper value.



So in reality, the memory is reclaimed and re-allocated for the new function, it reuse and override the values of old function. There is no actual cleanup in-between.

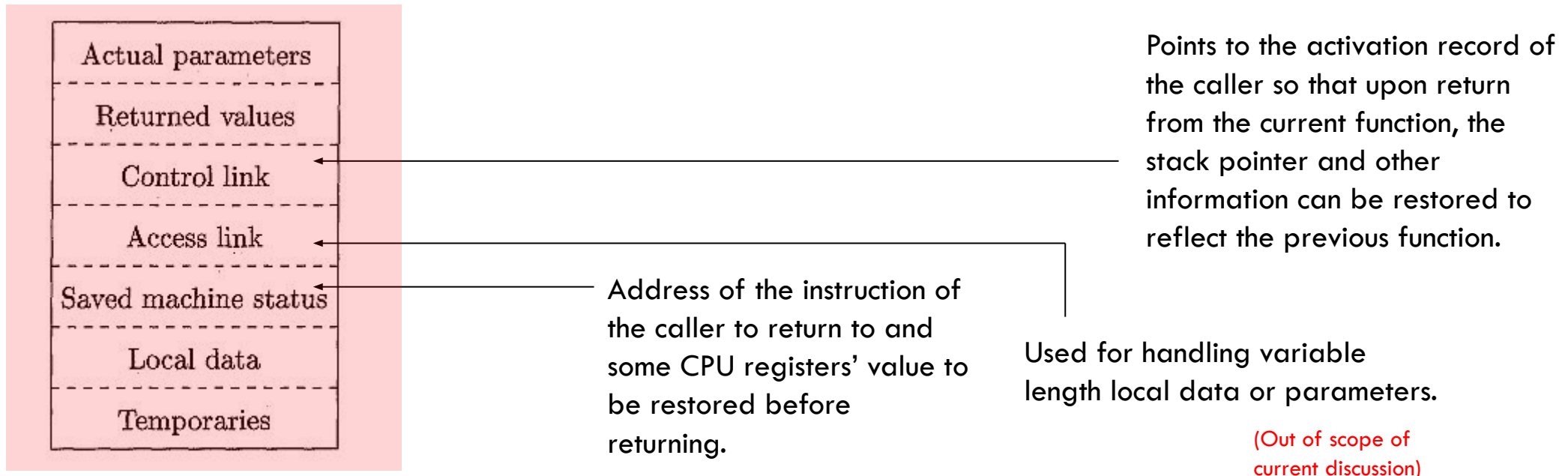
Simplified Stack Management

STACK ACTIVATION FRAME LAYOUT

Compilers follow a uniform layout for storing function call information in stack for all functions.

The uniformity makes code generation easier.

However, across programming languages the layout can differ.



Layout of a function activation record for C like languages.

SAVED MACHINE STATE DETAIL

Remember that CPU operations only happen on data stored in CPU registers – not on data stored in RAM addresses.

This is because accessing RAM is quite slow compared to registers inside the CPU. Furthermore, RAM is a separate chip that cannot participate in CPU data-path during arithmetic/logic operations.

Consequently, compilers try to keep results of operations and value loaded from RAM in registers as much as possible and do load/store with RAM infrequently.

In case of function calls, thus there is a convention for saving the contents of some of the register's previous values in stack before a called function can use those registers.

This set of registers that a callee function restores to old value forms the saved machine state.

Whatever callee saves here is part of saved machine state in RISC-V5

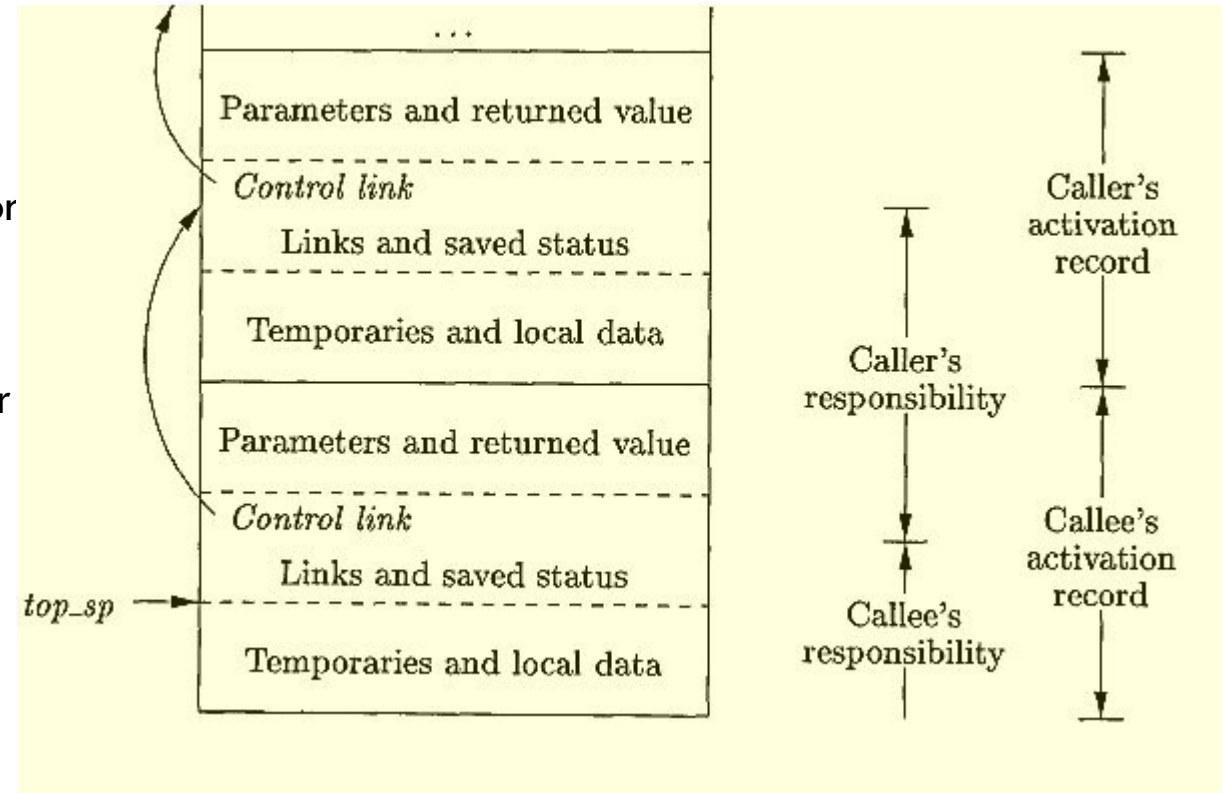
reg	name	saver	description
x0	zero		hardwired zero
x1	ra	caller	return address
x2	sp	callee	stack pointer
x3	gp		global pointer
x4	tp		thread pointer
x5-7	t0-2	caller	temporary registers
x8	s0/fp	callee	saved register / frame pointer
x9	s1	callee	saved register
x10-11	a0-1	caller	function arguments / return values
x12-17	a2-7	caller	function arguments
x18-27	s2-11	callee	saved registers
x28-31	t3-6	caller	temporary registers
pc			program counter

CALLER AND CALLEE RESPONSIBILITY BREAKDOWN

When compiler generates code for functions, it has to decide how to distribute the task of populating different parts of a function activation records between the caller function and the callee function.

Having the code in the callee allows the compiler to generate the code for a part only once to be used by many caller function's code.

However, things that the callee should not know beforehand must be populated by the caller.



IN ILLUSTRATION OF WHAT ACTUALLY HAPPENS

(CALLER'S SIDE)

```
void main() {  
    int x = 10;  
    int y = power(x, 5);  
    x = y - x  
}  
  
int power(int a, int b) {  
    int power = 1;  
    for (int i = 0; i < b; i++) {  
        power = power * a;  
    }  
    return power;  
}
```

Sample Source Code

<code>addi \$s0, 10</code>	Set the value of x to 10
<code>sw \$s0, \$sp[-4]</code>	Put parameters in stack
<code>swi 5, \$sp[-8]</code>	
<code>sw \$pc, \$sp[-16]</code>	Put control link in stack after reserving a spot for return value
<code>sw \$sp, \$sp[-20]</code>	Put main's stack pointer's address in stack
<code>addi \$sp, \$sp, -24</code>	Advance the stack pointers to the beginning of power's activation record
<code>Jal power_addr</code>	Jump to the address in the text section where power function is located
<code>lw \$s1, \$sp[-12]</code>	Retrieve the return value, 10^5 from stack and store it in the register for y
<code>sub \$s0, \$s1, \$s0</code>	Do $x = y - x$

Main function's code assuming \$s0 register storing the value of x and \$s1 the value of y.

IN ILLUSTRATION OF WHAT ACTUALLY HAPPENS

(CALLEE'S SIDE)

```
void main() {  
    int x = 10;  
    int y = power(x, 5);  
    x = y - x  
}  
  
int power(int a, int b) {  
    int power = 1;  
    for (int i = 0; i < b; i++) {  
        power = power * a;  
    }  
    return power;  
}
```

Sample Source Code

Note, some of my index calculation may be wrong, but I hope you get the general idea.

sw \$s0, \$sp[0] ...	Save machine status registers' values in stack
sw \$s11, \$sp[-44] addi \$sp, \$sp, -48	Advance the stack pointer
lw \$s0, \$sp[64] ls \$s1, \$sp[60]	Retrieve a and b from stack into \$s0 and \$s1 registers
addi \$s2, \$zero, 1 add \$s3, \$zero, \$zero loop_addr: bge \$s3, \$s1, loop_end mult \$s2, \$s2, \$s0 addi \$s3, \$s3, 1 jump loop_addr Loop_end:	Utilize \$s2 for power variable instead of any RAM address, similarly \$s3 for the index i.
sw \$s2, \$sp[56] lw \$s0, \$sp[44] ...	Store the result in space in stack designated by the caller
lw \$s11, \$sp[4] lw \$ra, \$sp[52] lw \$sp, \$sp[48] jr \$ra	Restore the machine state registers to their old values Retrieve the return address \$pc to jump back to Restore the stack pointer to the value assumed by the main function Go back to the specific line of the main just after the function call

Actual code of the callee, other things are just function call overhead.

PARTING MESSAGE

Function call is costly.

Thus, it is better not to create functions for small chunks of code.

However, modern compilers do machine dependent and independent optimizations to collapse function calls into equivalent code inside the caller function to avoid function call overheads.

This is called **inlining functions**.

Furthermore, instead of loading and storing local variables in stack, compilers, try to use the machine status registers as much as possible for local variables to avoid memory access instructions.